

Module - 6:

Domain Driven Design (DDD)

- Amjad Hossain
12th April, 2026

Why are we talking about DDD

The problem we face in real systems

- Code becomes “feature soup”: controllers → services → repositories with mixed business rules
- Business logic is scattered and duplicated
- Every change breaks something else (fear-driven development)
- Teams argue about terms because code doesn't match business language

Why are we talking about DDD

The problem we face in real systems

- Code becomes “feature soup”: controllers → services → repositories with mixed business rules
- Business logic is scattered and duplicated
- Every change breaks something else (fear-driven development)
- Teams argue about terms because code doesn't match business language

DDD is a way to reduce that chaos

- Put business rules in the center
- Make code speak the business language
- Split complexity into manageable parts

What DDD is (naive definition)

A way to design software around business reality

- First, learn the business domain deeply

What DDD is (naive definition)

A way to design software around business reality

- First, learn the business domain deeply
- Then model it in code so that:
 - business terms are clear
 - rules are enforceable
 - and changes are safer

When DDD is a good fit (and when not)

Good fit

- Complex business rules (banking, payments, RTGS, AML, KYC, limits, approvals)
- Many edge cases and exceptions
- Multiple teams working in parallel
- Simple Example:
 - Beneficiary Management System
 - Customer Onboarding Service
 - Transaction Limit Service

When DDD is a good fit (and when not)

Good fit

- Complex business rules (banking, payments, RTGS, AML, KYC, limits, approvals)
- Many edge cases and exceptions
- Multiple teams working in parallel
- Simple Example:
 - Beneficiary Management System
 - Customer Onboarding Service
 - Transaction Limit Service

Not ideal

- Simple CRUD apps with minimal rules
- Very short-lived prototypes
- Tiny projects where complexity is not in the domain
- Simple Example:
 - Auth Service
 - Notification Service
 - Reporting Service

Core DDD building blocks

Strategic DDD (macro)

- Domain, Subdomains
- Bounded Context
- Context Map
- Ubiquitous Language

Tactical DDD (micro)

- Entity, Value Object
- Aggregate & Aggregate Root
- Repository
- Domain Service
- Domain Event

Domain & Subdomain

Domain: the business area you're solving

Example: Banking System

Subdomain: a slice of that domain

Example subdomains:

- Payment
- Lending
- Treasury

Bounded Context

Bounded Context = a boundary where words mean one specific thing

- Inside the boundary: same rules, same meaning
- Outside: meaning can be different

Example:

- In “Beneficiary Context”, “Activation” means “approved and usable”
- In “Account Context”, “Activation” might mean “account opened”

Context Map (how contexts connect)

A **Bounded Context** is like a department that has:

- Its own rules
- Its own data
- Its own language

Now the question is:

When two departments talk to each other, how do they do it without breaking each other?

That is what a **Context Map** explains.

It shows:

- Who depends on whom
- Who controls the model
- Who adapts to whom

Ubiquitous Language

One language for business + engineers

- If business says “Beneficiary”, code should say “Beneficiary” too.
- Avoid vague tech words like:
 - Data, Record, DTO, TblBen
- Example glossary:
 - Beneficiary, Owner Customer, Activation, Verification, Maker-Checker, Limit

Entity vs Value Object

Entity

- Has identity (ID)
- Can change over time
- Example: Beneficiary (id = beneficiaryId)

Value Object

- No identity, defined by values
- Usually immutable
- Example: AccountNumber, MobileNumber, BankRoutingCode

A real-life analogy

- **Domain:** City Management
- **Subdomains:** Transport, Police, Utilities
- Each subdomain has its own rules and language
- If all departments share one “giant office”, everyone blocks everyone

DDD says: separate responsibilities clearly, but coordinate via agreed contracts.

Question 1: Why each subdomain is not a separate Domain?

Answer 1: A subdomain is not a separate domain because domain vs subdomain is relative to the scope of the business being modeled. If the focus of the system shifts such that a subdomain becomes the primary problem space, then from that new perspective it becomes the domain.

Question 2: Can each Subdomain be a different Microservice?

Answer 2: If the model inside a boundary:

- Has its own language
- Has its own data
- Has its own rules
- Can evolve independently

Then it is a candidate for a microservice. That boundary is a **Bounded Context**, not necessarily a Subdomain.

Let's Pick Up a simple service we defined earlier

RTGS-Return-Management-Service

is the simplest (and cleanest) one to introduce DDD step-by-step

Let's Pick Up a simple service we defined earlier

RTGS-Return-Management-Service

Why it beats the others for presenting DDD:

- It has **clear domain language** (Return, Rejection Reason, Mandated XML, Return Status).

Let's Pick Up a simple service we defined earlier

RTGS-Return-Management-Service

Why it beats the others for teaching DDD:

- It has **clear domain language** (Return, Rejection Reason, Mandated XML, Return Status).
- It has **strong business rules/invariants** but **low integration complexity** (compared to Posting/Transfer/CBS package).

Let's Pick Up a simple service we defined earlier

RTGS-Return-Management-Service

Why it beats the others for teaching DDD:

- It has **clear domain language** (Return, Rejection Reason, Mandated XML, Return Status).
- It has **strong business rules/invariants** but **low integration complexity** (compared to Posting/Transfer/CBS package).
- It naturally demonstrates DDD building blocks: **Aggregate, Value Objects, Domain Service, Domain Events, Repository, Application Service**.

Let's Pick Up a simple service we defined earlier

RTGS-Return-Management-Service

Why it beats the others for teaching DDD:

- It has **clear domain language** (Return, Rejection Reason, Mandated XML, Return Status).
- It has **strong business rules/invariants** but **low integration complexity** (compared to Posting/Transfer/CBS package).
- It naturally demonstrates DDD building blocks: **Aggregate, Value Objects, Domain Service, Domain Events, Repository, Application Service**.
- It avoids “false DDD” (wrapping pure integration work in domain terms), which happens with Transfer/Canonicalization/Facade.

RTGS-Return-Management-Service (DDD)

Question: what problem does this service solve?

When an incoming RTGS transaction is **not valid**, the system must:

- decide that it must be returned
- record **why** it is being returned
- generate a **return message/XML**
- track the return through statuses like draft, generated, sent, acknowledged

RTGS-Return-Management-Service (DDD)

Question: what problem does this service solve?

When an incoming RTGS transaction is **not valid**, the system must:

- decide that it must be returned
- record **why** it is being returned
- generate a **return message/XML**
- track the return through statuses like draft, generated, sent, acknowledged

Let's start with:

- what is a **Return**
- what rules does a Return follow
- what changes are allowed
- what words do business people use

RTGS-Return-Management-Service (DDD)

Ubiquitous Language for this service:

- | | |
|--|--|
| <ul style="list-style-type: none">● Return● Return Reason● Original Transaction● Return Status● Return Message | <ul style="list-style-type: none">● Acknowledgement● Correlation Reference● Generated At● Sent At● Acknowledged At |
|--|--|

RTGS-Return-Management-Service (DDD)

Bounded Context: Return Management

Inside this context:

- “**Return**” means a rejected RTGS item that must be sent back in the required format.
- “**Reason**” means the official rejection reason.
- “**Status**” means the lifecycle of that return.

RTGS-Return-Management-Service (DDD)

Bounded Context: Return Management

Inside this context:

- “**Return**” means a rejected RTGS item that must be sent back in the required format.
- “**Reason**” means the official rejection reason.
- “**Status**” means the lifecycle of that return.

Outside this context:

- **Validation** decides whether something is invalid.
- **Transfer** sends files.
- **Notification** sends SMS.

RTGS-Return-Management-Service (DDD)

Aggregate Root: RtgsReturn

Because the whole business process revolves around one return record and its lifecycle.

a “Return”:

- belongs to one original transaction/message
- has one reason
- has one lifecycle
- may have generated payload/content
- may be sent and acknowledged later

So the aggregate root is the main object that protects business rules

RTGS-Return-Management-Service (DDD)

What data belongs inside the Aggregate Root?

- | | |
|---|---|
| <ul style="list-style-type: none">• Return Id• Original Transaction Reference• Return Reason• Return Status• Created At | <ul style="list-style-type: none">• Generated At• Sent At• Acknowledged At• Return Payload Reference or XML content• Failure/Error note if needed |
|---|---|

RTGS-Return-Management-Service (DDD)

Which parts are Entities and which are Value Objects?

Entity

An **Entity** has identity.

Example:

- `RtgsReturn`

Even if its status changes from Draft to Generated to Sent, it is still the same return.

Value Object

A **Value Object** has no identity; it is defined by its value.

Good Value Objects here:

- `ReturnReason`
- `TransactionReference`
- `ReturnPayload`
- `CorrelationId`
- `ReturnStatus` can be an enum or richer value depending on complexity

Example:

Two `ReturnReason("Invalid Account Number")` are the same if their values are the same.

RTGS-Return-Management-Service (DDD)

Main business rules

DDD becomes useful when business rules live in the domain, not scattered in controllers or service layer.

For example:

- A return can only be created for a rejected/invalid transaction.
- A return must have a valid return reason.
- A return cannot be sent before payload is generated.
- A return cannot be acknowledged before it was sent.
- A sent return should not be generated again.
- Acknowledged return should not be modified.

These are **domain invariants**. The aggregate root should protect them.

RTGS-Return-Management-Service (DDD)

Simple lifecycle

- **Draft:** return exists, but XML not generated yet
- **Generated:** return XML/message ready
- **Sent:** transferred out
- **Acknowledged:** external party confirmed receipt
- **Failed:** generation or sending failed

This lifecycle belongs in the domain, not in UI logic.

RTGS-Return-Management-Service (DDD)

Sample Project structure:

<https://github.com/AmjadHossainRahat/sonali-intellect/blob/main/Session-3/DDD-Sample-codes/ProjectStructure.md>

Sample Project in Spring Boot:

<https://github.com/AmjadHossainRahat/sonali-intellect/tree/main/Session-3/DDD-Sample-codes/rtgs-return-management-springboot>

Thank you